

Beyond Document Grounding: Span-Level Hallucination Detection over Code, Tool Output, and Documents

Ádám Kovács¹, Bowei He^{2,3}, Xue Liu^{2,3}, István Boros¹, Szilveszter Tóth¹, Gábor Recski^{1,4}

¹KR Labs, ²MBZUAI, ³McGill University, ⁴TU Wien

Correspondence: kovacs@krlabs.eu

Abstract

Hallucination detection for retrieval-augmented generation (RAG) is usually evaluated on natural-language document evidence. However, grounded generation systems increasingly rely on structured inputs: source code, developer-tool output, markdown documents, tables, and repository metadata. We introduce a unified benchmark for span-level hallucination detection over code, tool output, structured documents, and existing natural-language RAG datasets. The benchmark is built by starting from grounded correct answers, injecting localized hallucinations with exact character labels, and validating the code test split with evidence-based review. Our fine-tuned Qwen3.5-2B detector reaches 0.689 span-F1 on the unified test set and 0.60 on the code-agent source, where it substantially outperforms LettuceDetect-large (0.17) and the strongest zero-shot LLM judges we evaluated (at most 0.22). The same model remains competitive on established natural-language benchmarks, with 81.8 RAGTruth example-F1 and 0.724 English PsiloQA IoU.

1 Introduction

Retrieval-augmented generation (RAG) grounds model outputs in external evidence (Lewis et al., 2020), but it does not remove the need for verification. A generated answer can still contradict the retrieved context, introduce unsupported information, or cite a reference that is not present in the evidence. Hallucination detection methods therefore ask whether an answer is supported by the supplied context, often at the level of examples, sentences, tokens, or spans (Niu et al., 2024; Rykov et al., 2025; Vazquez et al., 2025).

Most existing benchmarks and detectors study this problem in natural-language RAG, where both

the evidence and the answer are usually document text (Niu et al., 2024; Belyi et al., 2025; Tang et al., 2024; Kovács and Recski, 2025; Rykov et al., 2025; Vazquez et al., 2025). Real grounded-generation systems are broader: coding agents work over repository files, git history, and test output (Jimenez et al., 2024); developer assistants summarize command output and tool observations (Kovacs, 2026); and research or documentation systems retrieve markdown pages, tables, citations, and structured documents (Recski et al., 2026; Index, 2026). These settings are not well covered by current training data or evaluations: there is no shared span-level benchmark that covers generated code, tool observations, and structured documents under the same verification task.

We study post-generation verification for this structured setting: given an answer that has already been produced, together with its request and context, a detector should flag the parts that the evidence does not support. We frame this at the span level rather than as a whole-answer accept/reject decision, because in code and tool output a single unsupported substring, such as a wrong field, a fabricated method name, a misreported value, or an invented section reference, can change program behavior or mislead a user while leaving the rest of the answer correct. A verifier should therefore point to the unsupported substring, not just reject the answer.

Existing hallucination-detection benchmarks and models leave this setting only partially covered. RAGTruth (Niu et al., 2024), Luna (Belyi et al., 2025), MiniCheck (Tang et al., 2024), and LettuceDetect (Kovács and Recski, 2025) verify generated text against retrieved documents. Code hallucination work studies generated snippets (Tian

et al., 2025; Agarwal et al., 2025), generation-time divergence (Jiang et al., 2024), or agent trajectories (Liu et al., 2026). These resources are useful, but they do not provide one span-level formulation that covers natural-language RAG, structured documents, source code, and developer-tool output.

We address this gap with a unified benchmark spanning SWE-bench code (Jimenez et al., 2024), developer-tool output from Squeez (Kovacs, 2026), ACL paper chunks (Recski et al., 2026), READMEs, Wikipedia markdown (Index, 2026), RAGTruth (Niu et al., 2024), and PsiloQA (Rykov et al., 2025). The dataset construction uses a shared edit-based labeling step: start from a correct grounded answer, inject a small localized hallucination, recover exact character offsets from the edit, and split by grounding source so the test set uses unseen repositories, papers, or articles. On this benchmark we train two detector families. Our best model is `LettuceDetect-Qwen-2B`, a fine-tuned Qwen3.5-2B detector (Qwen Team, 2026a) with a 32,768-token maximum sequence length that localizes unsupported spans across code, tool output, structured documents, and natural-language RAG. We also train `LettuceDetect-mmBERT-base`, a 307M-parameter mmBERT-base encoder (Marone et al., 2025), as a token classification model. Multilingual supervision comes from the 14-language PsiloQA portion of the training set.

Our contributions are:

- a span-level task formulation for post-generation hallucination detection over structured grounded-generation contexts;
- a unified benchmark with 74,285 newly constructed examples across code, tool output, and structured documents, plus converted examples from existing natural-language RAG benchmarks;
- two detector families that localize unsupported spans across code, tool output, structured documents, and natural-language RAG: `LettuceDetect-Qwen-2B`, a fine-tuned Qwen3.5-2B detector, and `LettuceDetect-mmBERT-base`, a 307M-parameter encoder baseline, with results showing that the generative detector substantially outperforms off-the-shelf detectors and zero-shot LLM judges on the code-agent split while remaining competitive on natural-language RAG.

Code, data, model checkpoints, prompts, evaluation outputs, and the reviewed code-test arbitration files are released through GitHub and Hugging Face.¹

2 Related Work

Grounded text verification. Hallucination detection in grounded generation includes prompt-based judging, self-consistency methods such as Self-CheckGPT (Manakul et al., 2023), and benchmark-driven detectors trained on datasets such as HaluEval (Li et al., 2023) and RAGTruth (Niu et al., 2024). Recent compact detectors, including Luna (Belyi et al., 2025) and LettuceDetect (Kovács and Recski, 2025), show that long-context encoders can localize unsupported spans at lower cost than LLM judges. We use mmBERT, a ModernBERT-family multilingual encoder trained with annealed language learning (Marone et al., 2025); PsiloQA reports strong span-localization results from fine-tuning mmBERT-base (Rykov et al., 2025). Other work optimizes related but different objectives: RAG-HAT (Song et al., 2024) reports response-level F1 on RAGTruth, RL4HS (Su et al., 2025) optimizes span-F1 with reinforcement learning, and PsiloQA (Rykov et al., 2025) and Mu-SHROOM (Vazquez et al., 2025) evaluate multilingual span localization. Fine-grained taxonomies have also been proposed, most notably FAVA (Mishra et al., 2024), but these taxonomies are still mainly designed for natural-language responses grounded in textual documents.

Code hallucination. CodeHalu (Tian et al., 2025) and CodeMirage (Agarwal et al., 2025) treat hallucinations as defects in generated snippets. Collu-Bench (Jiang et al., 2024) predicts hallucination during generation from token probabilities and execution feedback. AgentHallu (Liu et al., 2026) attributes failures across agent trajectories. Delulu (Erfanian et al., 2026) is closest in spirit because it targets code hallucination, but it is an execution-verified fill-in-the-middle benchmark with binary accept/reject labels. In contrast, our task is post-generation, repository-grounded, and span-level.

3 Task

Each example consists of a request q , context c , and answer a . The context may contain source

¹Code at <https://github.com/KRLabsOrg/LettuceDetect>; models and datasets under <https://huggingface.co/KRLabsOrg>

code at a specific commit, tool output, or structured document text. The goal is to predict character spans in a that are not supported by q and c .

We use three top-level hallucination categories. **Contradiction** covers wrong logic, values, fields, or conditions in otherwise plausible code. **Unsupported addition** covers extra behavior or claims not requested or evidenced. **Fabricated reference** covers invented methods, attributes, keyword arguments, sections, or identifiers. This split follows the common distinction made by prior taxonomies between conflicts with evidence, baseless additions, and invented entities or references (Niu et al., 2024; Mishra et al., 2024). For code, the first two are mostly judged against the request, while fabricated references are judged against repository and library evidence. The detector does not see generator logs or tool trajectories; it only sees inputs available to an external checker after the answer has been produced.

The top-level labels are paired with 13 subcategories that describe the surface element affected: entity, temporal, numerical, value, relational, identifier, section, attribute, claim, behavior, elaboration, subjective, and unspecified. We choose these subcategories by harmonizing distinctions used in prior taxonomies: RAGTruth’s conflict and baseless-information labels (Niu et al., 2024), FAVA’s entity, relation, invented, subjective, and unverifiable labels (Mishra et al., 2024), code-hallucination categories such as naming, resource, and logic errors (Tian et al., 2025), and the code/tool labels needed by our structured sources. The result keeps the three-way category decision interpretable while preserving the surface-level error types needed for analysis.

4 Benchmark Construction

4.1 Sources

The benchmark has five newly constructed sources and two incorporated natural-language RAG benchmarks. The programming-oriented sources are *code*, built from SWE-bench (Jimenez et al., 2024), and *tool output*, built from Squeez (Kovacs, 2026) containing a query, verbose tool observation, and gold relevant lines. The structured-document sources are *ACL*, built from ACL-Verbatim retrieved paper chunks and questions (Recski et al., 2026); *README*, collected

Source	Modality	Samples	Halluc.
Code (SWE-bench)	code	18,524	9,268
Tool output	tool output	11,365	5,682
Papers (ACL)	markdown	5,355	2,677
READMEs	markdown	13,803	6,900
Wikipedia	markdown	25,238	12,618
Total		74,285	37,145

Table 1: Newly constructed benchmark sources. The complete evaluation also includes converted RAGTruth and PsiloQA examples. New-source splits are train/dev/test = 66,368 / 2,816 / 5,101.

from popular GitHub repositories through the GitHub API; and *Wikipedia*, sampled from English open-wikipedia-markdown articles (Index, 2026). We also include RAGTruth, a human-annotated word-level hallucination corpus for RAG outputs (Niu et al., 2024), and PsiloQA, a 14-language span-level hallucination benchmark built from Wikipedia QA (Rykov et al., 2025), to keep the model tied to established natural-language detection tasks.

All new sources use the same sample abstraction: a prompt containing context and request, an answer, and character-level span annotations over the answer. Context and clean-answer construction differ by source. Code examples use the gold SWE-bench fix; tool-output examples generate a short answer from the Squeez query and relevant lines; ACL examples use the top five retrieved paper chunks as context; README and Wikipedia examples are generated from heading-based markdown chunks. Train, development, and test splits are separated by grounding source.

4.2 Code Source

SWE-bench provides real GitHub issues, repository metadata, base commits, and gold patches. For each instance we recover the files touched by the gold patch at the base commit and render the gold fix as one coding-assistant answer: a patched function, a changed-line fragment, or a natural-language edit instruction. Clean examples use this answer verbatim. Hallucinated examples contain a small edit to the same answer. We do not include clean and hallucinated versions of the same instance as a pair. This setup differs from snippet-only code hallucination, because the answer is evaluated against a concrete repository state and request.

The three answer renderings are meant to cover the kinds of outputs a developer may ask an assis-

tant for. The *function* rendering gives the largest patched function that fits the length cap, and is closest to a direct code suggestion. The *fragment* rendering gives the changed hunk, preserving the local edit without forcing the model to read a full function. The *edit* rendering gives an instruction such as “in file X, replace Y with Z”, which is common when agents summarize a patch rather than printing a full diff. Instances whose gold answer is a trivial version bump or too long for the context budget are skipped. This filtering is practical rather than conceptual: if the answer itself consumes the full sequence window, the detector cannot use the repository evidence.

The raw SWE-bench issue text is also rewritten by our pipeline into a short developer request. We keep the technical intent but remove issue-tracker noise, reproduction logs, and long discussions that would make the answer-verification task depend on irrelevant text. This request is included in the detector input. It matters especially for `unsupported_addition`, because an extra behavior can be unsupported even when it is syntactically valid and uses real repository symbols.

4.3 Generation Pipeline

The construction has a shared labeling step across sources. First, source-specific preparation builds the context, request, and a known-correct grounded answer. Then a source-specific injector proposes localized replacement edits as structured `original/hallucinated` pairs (Figure 1). Applying these edits gives exact character offsets without diffing mixed natural-language/code answers.

The edit-based design is important for label quality. Instead of diffing two mixed natural-language/code answers, we apply the injector’s structured replacement and locate the hallucinated substring directly in the final answer. For the code source, the clean answer is the gold SWE-bench fix and the injector is Gemma 4 31B (Google DeepMind, 2026). For tool output and the markdown sources, Qwen 3.6 35B A3B (Qwen Team, 2026b) first generates a request or clean answer when the source does not already supply one, and then proposes the hallucination edit. Because the final label comes from the applied replacement, the span can stay narrow: in `torch.cuda.set_active_device(gpu)`, only `set_active_device` is labeled, not the

surrounding correct call structure.

The injector is prompted to preserve the surrounding answer and to make only localized changes. For code, the target changes fall into two families. Intent errors change a value, field, condition, or side effect while leaving the answer plausible. Structural errors replace a real method, attribute, or keyword with a plausible name that does not exist. The injector returns JSON containing the original substring, the hallucinated substring, and the category. We then apply the replacement deterministically. Attempts are discarded when the original substring is not uniquely found, the edit is a no-op, the hallucinated span covers too much of the answer, or the injected name already occurs in the evidence.

This approach deliberately trades generation yield for cleaner labels. Roughly half of the attempted code injections pass the automatic checks. The most common failures are non-unique originals, overly broad edits, hint words such as “incorrect”, and fabricated references that are not actually absent from context. These rejections are useful: they remove examples where the model could learn artifacts of the generation process rather than the verification task.

Reference grounding. Clean answers often refer to sibling methods, imported helpers, or third-party APIs not present in a truncated source context. If these are not added, a verifier cannot distinguish a correct but unseen reference from a fabricated one. We therefore append referenced definitions from the modified files, imported repository modules, and modules imported by the changed files, all resolved at the historical base commit rather than by current-code search. For third-party APIs, which are not in the repository at all, we retrieve real signatures from Context7², a library-documentation index: we parse the external libraries imported by the answer and query Context7 for each imported symbol’s signature and usage, then append the returned snippets (Figure 3). This gives the verifier genuine evidence for external calls, so a real but unfamiliar library API is not penalized as a fabrication.

Other sources. The non-code sources use the same edit-application framework but different prompts. Tool-output injections misreport identifiers, line references, values, or claims from the

²<https://context7.com>

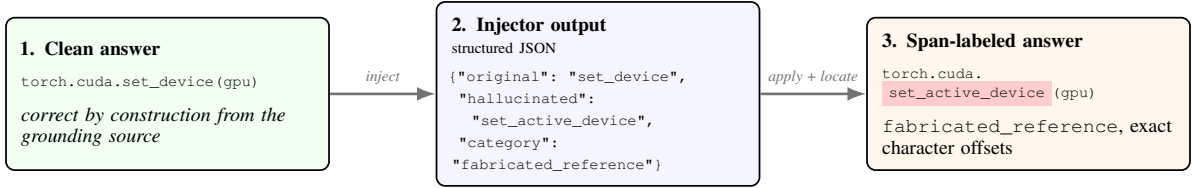


Figure 1: Edit-based injection yields exact spans. The injector returns each change as an original/hallucinated pair; applying the edit deterministically locates the hallucinated substring in the final answer.

Qualitative benchmark example: contradiction / wrong implementation

Repository / request. matplotlib__matplotlib-23049. User request: “Can you add support for a minor keyword argument to plt.xticks()?”

Answer snippet.

```
def xticks(ticks=None, labels=None, *, minor=False, **kwargs):
    ...
    if ticks is None:
        locs = ax.get_xticks(minor=minor)
    else:
        [H] locs = ax.set_xticks(ticks, minor=not minor) [/H]
    ...

def yticks(ticks=None, labels=None, *, minor=False, **kwargs):
    ...
    if ticks is None:
        [H] locs = ax.get_yticks(minor=not minor) [/H]
    ...
```

Type. Contradiction (wrong implementation). The answer is syntactically valid and locally plausible, but reverses the intended behavior of the `minor` argument.

Figure 2: Example benchmark instance. We use [H] ... [/H] markers for gold unsupported spans.

observation. ACL injections use paper-specific numerical, entity, relational, methodological, and citation-like edits detectable from the retrieved excerpts. README and Wikipedia injections use a generic markdown prompt covering numerical, temporal, entity, relational, fabricated-reference, and unsupported-claim edits.

4.4 Test-Set Verification

For quality assurance, we reviewed every code-source test sample before release: 2,038 examples were reviewed and 2,015 retained. Model-assisted triage first flagged span validity, category, boundary, and plausibility issues³; flagged cases were then re-judged blindly and finally arbitrated by the authors against the true pre-fix repository evidence rather than the injector’s intended edit. For the 44 disputed hallucinations, arbitration upheld 41 as genuine and dropped 3 that matched original code. The review tightened 235 boundaries, dropped 23

invalid spans, corrected 2 categories, reclassified 5 examples as clean, and removed 23 examples for question–answer mismatch, no-op edits, or incoherent rendering. No rebalancing was applied after review; the released code test set is 50.3% hallucinated. Per-sample verdicts, contested cases, resolutions, and the rubric are released with the dataset.

5 Benchmark Characteristics

The code source contains 18,524 examples from 53 repositories, split by SWE-bench repository into 16,319 train / 190 development / 2,015 test examples over 35 / 6 / 12 disjoint repositories (50.0% hallucinated overall). Average answers are about 1.8k characters, and hallucinated code examples contain just under two labeled spans on average. Answer formats are uneven—function (8,550), fragment (4,893), and edit-style (5,081) renderings—as are error types over hallucinated examples: contradiction (4,719), fabricated reference (2,275), and unsupported addition (2,274).

³We used Claude Sonnet 4.6 to flag candidate issues; final decisions were made by the authors during arbitration.

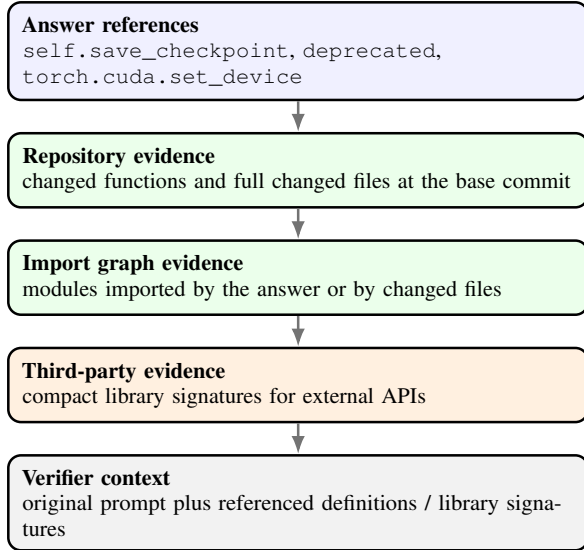


Figure 3: Reference grounding for code examples. Correct references missing from the truncated source context are resolved at the base commit and appended to the verifier context; third-party APIs are grounded with compact signatures.

The examples vary in what evidence is needed. Some hallucinations are local: a wrong config key is contradicted by the nearby code. Others require cross-file evidence, for example a method called on `self` whose definition lives in an imported mixin. Third-party API fabrications require library signatures rather than repository code. Without reference grounding, correct answers would contain many unsupported-looking names simply because the context was truncated.

For the tool-output dataset the context is often an observation rather than a source file: a test failure, a grep result, a package-manager response, or a git command. Here hallucinations are usually value and relation errors: reporting the wrong file, count, version, status, or failing test. The markdown sources broaden the data beyond code while preserving structure. ACL papers, READMEs, and Wikipedia pages contain headings, tables, citations, and lists, so the detector sees contexts that are not plain paragraphs but are still text-grounded.

6 Experiments

6.1 Models and Metrics

Our best model, `LettuceDetect-Qwen-2B`, is a Qwen3.5-2B generative detector (Qwen Team, 2026a) fine-tuned to return JSON spans with category and subcategory. We fine-tune and evaluate it with a 32,768-token maximum sequence

length, so each prediction can include the user request, retrieved documents or repository evidence, tool output, and the answer to check. It is trained on the unified training set using the same data-agnostic prompt across sources. We compare it with `LettuceDetect-mmBERT-base`, a 307M-parameter mmBERT-base encoder (Marone et al., 2025) trained on the same task, an LFM2.5-8B-A1B generative sibling (Liquid AI, 2026) fine-tuned with the same prompt, `LettuceDetect-large` (Kovács and Recski, 2025), answer-level faithfulness detectors, and two large zero-shot LLM judges: Nemotron-3-Ultra-550B (NVIDIA et al., 2026) and gpt-oss-120b (OpenAI et al., 2025).

We report character-overlap span precision, recall, and F1; example-F1, where an answer is flagged if at least one span is predicted; and mean IoU between predicted and gold span mass. For typed detection, a predicted span only receives credit when its category or subcategory also matches.

Training details. The generative detector is trained with supervised fine-tuning on the merged structured and natural-language training split. The 74,285 examples in Table 1 are newly constructed; after adding converted RAGTruth and PsiloQA examples, the full split contains 145,250 train, 6,171 validation, and 10,698 test examples. The training and evaluation splits are released on Hugging Face, and the construction/evaluation code is released on GitHub.⁴ We fine-tune Qwen3.5-2B with LoRA rank 32, $\alpha = 64$, dropout 0, bf16 weights, learning rate 2×10^{-4} , linear schedule with 3% warmup, weight decay 0.01, two epochs, effective batch size 32, and a 32,768-token maximum sequence length. The multilingual portion of the training data comes from PsiloQA, while the long context budget is mainly used by code, tool-output, and structured-document examples. The prompt is the same across sources: it defines a hallucinated span, lists the taxonomy, then provides the user request, context, and answer to verify. Returned strings are matched back into the answer to recover character offsets.

`LettuceDetect-mmBERT-base` follows the `LettuceDetect` token-classification architecture with `jhu-clsp/mmBERT-base` as the backbone: context and request tokens are masked in the loss, and answer tokens receive supported/unsup-

⁴Code at <https://github.com/KRLabsOrg/LettuceDetect>; models and datasets under <https://huggingface.co/KRLabsOrg>

ported labels. We train it on the same unified split with an 8,192-token maximum sequence length, batch size 8, gradient accumulation 4, learning rate 10^{-5} , and three epochs, selecting the best checkpoint by development hallucinated-token F1. The encoder is cheaper at inference and decodes contiguous positive answer tokens into character spans, but it only predicts binary spans in its base form. We also evaluate a typed encoder cascade in which spans from the frozen binary model are classified by a label-conditioned head that scores each span against category descriptions.

6.2 Main Results

Table 2 gives per-source results for the 2B generative detector. The model performs best on README and Wikipedia, where grounding resembles factual document QA, and remains strong on the harder coding-agent sources. Code-agent is the most difficult source: the context is long, the answer often contains new code, and some errors are intent mistakes rather than simple factual contradictions.

The generative detector outperforms LettuceDetect-mmBERT-base on every source: 0.689 vs. 0.642 span-F1 overall, and 0.602 vs. 0.508 on code-agent. On code-agent answers it is also well above LettuceDetect-large and the strongest zero-shot LLM judge we evaluated; on natural-language RAG benchmarks it is close to specialized systems.

6.3 Comparison with Existing RAG Benchmarks

Table 3 compares against published natural-language RAG results. On RAGTruth, our model is highly competitive with specialized systems while also covering code and tool output. On PsiloQA, it sets the best reported English IoU (0.724), above the fine-tuned mmBERT-base model (0.707) of the benchmark authors and far above their 32B few-shot judge (0.400). Across all 14 PsiloQA languages, it reaches 0.689 mean IoU, compared with 0.623 for the PsiloQA mmBERT-base model and 0.383 for the Qwen2.5-32B judge reported by Rykov et al. (2025). These are the strongest reported results among the published PsiloQA systems we compare against, while using the same detector trained for code, tool output, and structured documents.

6.4 Code and Tool Evidence Evaluation

Table 4 shows results on the code-agent test set. LettuceDetect-large, trained for natural-language RAG, reaches only 0.17 span-F1. Large zero-shot judges find some true error regions but over-predict heavily: the task-aware Nemotron prompt improves over a generic prompt but still reaches only 0.22 span-F1. Answer-level faithfulness systems, including HHEM-2.1-Open (Li et al., 2024), Lynx-8B (Ravi et al., 2024), Granite-Guardian-4.1-8B (Padhi et al., 2025), and MiniCheck-7B (Tang et al., 2024), show the same tendency at answer level: high recall but much lower precision on the hallucinated class. In most cases these judges flag correct newly written patch code as unsupported instead of checking whether the edit follows from the request and repository evidence.

Table 2 shows results on other sources. Under the same generic prompt, gpt-oss-120b reaches reasonable span-F1 on document-like data (ACL, README, Wikipedia), but performs poorly on tool output and code-agent answers (OpenAI et al., 2025).

6.5 Typed Detection

The generative detector emits a category and subcategory with every span. On the full unified test set it reaches category-gated span-F1 0.585 and subcategory-gated span-F1 0.468, compared with binary span-F1 0.689. Subcategory prediction is harder, especially for natural-language examples where `claim`, `elaboration`, `value`, and `relational` can overlap. As a cheaper alternative, we also evaluate a typed encoder cascade: given gold spans, the label-conditioned head reaches 0.82 category and 0.64 subcategory accuracy, but end-to-end it trails the generative model (category-gated span-F1 0.461 vs. 0.585, subcategory-gated 0.315 vs. 0.468 overall).

6.6 Error Analysis

The hardest remaining code examples are request-grounded intent errors and broad generated edits. A wrong field name or condition may use real repository symbols and look structurally valid, while only one small part of a larger generated block is unsupported. Tool-output examples show the same pattern when an otherwise fluent answer copies one version, count, filename, or status incorrectly.

Zero-shot judges make similar errors. With a generic prompt, Nemotron-3-Ultra often marks

Source	n	LettuceDetect-Qwen-2B				LD-mmBERT	LFM-8B	gpt-oss
		P	R	F1	Ex.-F1	F1	F1	F1
ALL	10,698	0.677	0.701	0.689	0.921	0.642	0.650	–
Code-agent	2,015	0.596	0.609	0.602	0.835	0.508	0.507	0.177
Tool output	617	0.737	0.702	0.719	0.907	0.588	0.692	0.331
ACL	440	0.752	0.745	0.749	0.942	0.579	0.670	0.602
README	641	0.889	0.844	0.866	0.984	0.751	0.804	0.666
Wikipedia	1,388	0.836	0.800	0.817	0.974	0.708	0.789	0.658
PsiloQA	2,897	0.702	0.765	0.732	0.966	0.714	0.714	–
RAGTruth	2,700	0.601	0.548	0.574	0.818	0.528	0.557	–

Table 2: Per-source results. P/R/F1 are character-overlap span metrics and Ex.-F1 is example-level F1 for LettuceDetect-Qwen-2B; LD-mmBERT, LFM-8B, and gpt-oss report span-F1. gpt-oss was run only on the five newly constructed sources.

Benchmark	Method	Score
RAGTruth Ex.-F1	RAG-HAT 8B (Song et al., 2024)	83.9
RAGTruth Ex.-F1	LettuceDetect-Qwen-2B	81.8
RAGTruth Ex.-F1	LettuceDetect-large (Kovács and Reeski, 2025)	79.2
RAGTruth Ex.-F1	Fine-tuned Llama-2-13B (Niu et al., 2024)	78.7
RAGTruth Ex.-F1	GPT-4 (Niu et al., 2024)	63.4
PsiloQA EN IoU	LettuceDetect-Qwen-2B	0.724
PsiloQA EN IoU	mmBERT-base (PsiloQA) (Rykov et al., 2025)	0.707
PsiloQA EN IoU	Qwen2.5-32B judge (Rykov et al., 2025)	0.400
PsiloQA 14-lang IoU	LettuceDetect-Qwen-2B	0.689
PsiloQA 14-lang IoU	mmBERT-base (PsiloQA) (Rykov et al., 2025)	0.623
PsiloQA 14-lang IoU	Qwen2.5-32B judge (Rykov et al., 2025)	0.383

Table 3: Comparison with established natural-language RAG benchmarks. Rows combine published numbers with our evaluation, so they should be read as external reference points rather than a single shared leaderboard.

clean patch code as fabricated when it should verify newly written lines against the request and repository evidence. A task-aware prompt improves precision from 0.11 to 0.13 but does not solve the problem, and gpt-oss-120b behaves similarly. In a small reasoning-judge diagnostic, correct patch lines were treated as unsupported, truncated context as evidence for fabricated references, and edit-style answers produced non-verbatim spans. These results show that strong zero-shot judges do not replace task-specific training here.

7 Conclusion

We introduced a span-level hallucination detection benchmark across code, tool output, structured documents, and natural-language RAG. A 2B generative detector reaches 0.689 span-F1 overall and 0.602 on code-agent answers, well above the off-the-shelf detectors and zero-shot LLM judges we evaluated. It also reaches the best reported English PsiloQA IoU (0.724) and 81.8 RAGTruth example-F1, close to RAG-HAT’s 83.9.

Detector (code-agent)	Eval	P	R	F1	Ex.-F1
LettuceDetect-large (v1)	span	0.112	0.373	0.172	0.684
Nemotron-3-Ultra, generic	span	0.108	0.665	0.186	0.655
Nemotron-3-Ultra, task-aware	span	0.132	0.589	0.216	0.700
gpt-oss-120b, task-aware	span	0.135	0.501	0.212	0.691
HHEM-2.1-Open	answer	0.50	0.86	0.63	–
Lynx-8B	answer	0.52	0.73	0.61	–
Granite-Guardian-4.1-8B	answer	0.53	0.90	0.66	–
MiniCheck-7B	answer	0.50	1.00	0.67	–
LettuceDetect-mmBERT-base	span	0.619	0.430	0.508	0.770
LFM2.5-8B-A1B (ours, SFT)	span	0.531	0.485	0.507	0.811
LettuceDetect-Qwen-2B	span	0.596	0.609	0.602	0.835

Table 4: Baselines on the code-agent test set. Span systems report character-overlap span P/R/F1 plus example-F1; answer systems report hallucinated-class example P/R/F1.

8 Limitations

Most labels come from synthetic injection. The reviewed code test set gives us confidence in that split, but train/development labels and non-code test labels are generated labels guarded by automated checks, and the code review is model-assisted rather than independently annotated by multiple human annotators. The benchmark covers final-answer verification, not full agent trajectories, and should not be read as measuring real-world hallucination prevalence. Because most errors were deliberately injected, the detector may learn patterns specific to how those errors were created. Further evaluation is needed to determine how well it detects naturally occurring errors in coding and tool-use systems.

References

Vibhor Agarwal, Yulong Pei, Salwa Alamir, and Xiaomo Liu. 2025. [Codemirage: Hallucinations in code generated by large language models](#). *Preprint*, arXiv:2408.08333.

- Masha Belyi, Robert Friel, Shuai Shao, and Atindriyo Sanyal. 2025. [Luna: A lightweight evaluation model to catch language model hallucinations with high accuracy and low cost](#). In *Proceedings of the 31st International Conference on Computational Linguistics: Industry Track*, pages 398–409, Abu Dhabi, UAE. Association for Computational Linguistics.
- Mahdi Erfanian, Nelson Daniel Troncoso, Aashna Garg, Amabel Gale, Xiaoyu Liu, Pareesa Ameneh Golnari, and Shengyu Fu. 2026. [Delulu: A verified multilingual benchmark for code hallucination detection in fill-in-the-middle tasks](#). *Preprint*, arXiv:2605.07024.
- Google DeepMind. 2026. Gemma 4 31b it model card. <https://huggingface.co/google/gemma-4-31B-it>.
- Open Index. 2026. [Open wikipedia \(markdown\)](#).
- Nan Jiang, Qi Li, Lin Tan, and Tianyi Zhang. 2024. [Collu-bench: A benchmark for predicting language model hallucinations in code](#). *Preprint*, arXiv:2410.09997.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [Swe-bench: Can language models resolve real-world github issues?](#) *Preprint*, arXiv:2310.06770.
- Adam Kovacs. 2026. [Squeeze: Task-conditioned tool-output pruning for coding agents](#). *Preprint*, arXiv:2604.04979.
- Ádám Kovács and Gábor Recski. 2025. [Lettucedetect: A hallucination detection framework for RAG applications](#). *Preprint*, arXiv:2502.17125.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Red Hook, NY, USA. Curran Associates Inc.
- Junyi Li, Xiaoxue Cheng, Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2023. [HaluEval: A large-scale hallucination evaluation benchmark for large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6449–6464, Singapore. Association for Computational Linguistics.
- Miaoran Li, Rogger Luo, and Ofer Mendelevitch. 2024. [HHEM-2.1-Open](#).
- Liquid AI. 2026. Lfm2.5-8b-a1b: Personal assistant on your laptop. *Liquid AI Blog*. www.liquid.ai/blog/lfm2-5-8b-a1b.
- Xuannan Liu, Xiao Yang, Zekun Li, Peipei Li, and Ran He. 2026. [Agenthallu: Benchmarking automated hallucination attribution of LLM-based agents](#). *Preprint*, arXiv:2601.06818.
- Potsawee Manakul, Adian Liusie, and Mark Gales. 2023. [SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017, Singapore. Association for Computational Linguistics.
- Marc Marone, Orion Weller, William Fleshman, Eugene Yang, Dawn Lawrie, and Benjamin Van Durme. 2025. [mmbert: A modern multilingual encoder with annealed language learning](#). *Preprint*, arXiv:2509.06888.
- Abhika Mishra, Akari Asai, Vidhisha Balachandran, Yizhong Wang, Graham Neubig, Yulia Tsvetkov, and Hannaneh Hajishirzi. 2024. [Fine-grained hallucination detection and editing for language models](#). *Preprint*, arXiv:2401.06855.
- Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. 2024. [RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10862–10878, Bangkok, Thailand. Association for Computational Linguistics.
- NVIDIA, Aaron Blakeman, Aaron Thomas, Aastha Jhunjhunwala, Abhibha Gupta, Abhinav Khattar, Adam Rajfer, Adi Renduchintala, Adil Asif, Aditya Vavre, Adriana Flores Miranda, Ahmad Bilal, Aileen Zaman, Ajay Hotchandani, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, Alex Gronskiy, Alex Kondratenko, and 554 others. 2026. [Nemotron 3 ultra: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning](#). *Preprint*, arXiv:2606.15007.
- OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, and 107 others. 2025. [gpt-oss-120b & gpt-oss-20b model card](#). *Preprint*, arXiv:2508.10925.
- Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhjit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehl, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, and 3 others. 2025. [Granite guardian: Comprehensive LLM safeguarding](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human*

Language Technologies (Volume 3: Industry Track), pages 607–615, Albuquerque, New Mexico. Association for Computational Linguistics.

Qwen Team. 2026a. [Qwen3.5: Accelerating productivity with native multimodal agents](#).

Qwen Team. 2026b. [Qwen3.6-35B-A3B: Agentic coding power, now open to all](#).

Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela, and Rebecca Qian. 2024. [Lynx: An open source hallucination evaluation model](#). *Preprint*, arXiv:2407.08488.

Gabor Recski, Szilveszter Toth, Nadia Verdha, Istvan Boros, and Adam Kovacs. 2026. [ACL-verbatim: Hallucination-free question answering for research](#). *Preprint*, arXiv:2605.21102.

Elisei Rykov, Kseniia Petrushina, Maksim Savkin, Valerii Olisov, Artem Vazhentsev, Kseniia Titova, Alexander Panchenko, Vasily Konovalov, and Julia Belikova. 2025. [When models lie, we learn: Multilingual span-level hallucination detection with PsiloQA](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 11663–11682, Suzhou, China. Association for Computational Linguistics.

Juntong Song, Xingguang Wang, Juno Zhu, Yuanhao Wu, Xuxin Cheng, Randy Zhong, and Cheng Niu. 2024. [RAG-HAT: A hallucination-aware tuning pipeline for LLM in retrieval-augmented generation](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1548–1558, Miami, Florida, US. Association for Computational Linguistics.

Hsuan Su, Ting-Yao Hu, Hema Swetha Koppula, Kundan Krishna, Hadi Pouransari, Cheng-Yu Hsieh, Cem Koc, Joseph Yitan Cheng, Oncel Tuzel, and Raviteja Vemulapalli. 2025. [Learning to reason for hallucination span detection](#). *Preprint*, arXiv:2510.02173.

Liyan Tang, Philippe Laban, and Greg Durrett. 2024. [MiniCheck: Efficient fact-checking of LLMs on grounding documents](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8818–8847, Miami, Florida, USA. Association for Computational Linguistics.

Yuchen Tian, Weixiang Yan, Qian Yang, Xuandong Zhao, Qian Chen, Wen Wang, Ziyang Luo, Lei Ma, and Dawn Song. 2025. [Codehalu: investigating code hallucinations in llms via execution-based verification](#). In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’25/IAAI’25/EAAI’25. AAAI Press.

Raul Vazquez, Timothee Mickus, Elaine Zosa, Teemu Vahtola, Jörg Tiedemann, Aman Sinha, Vincent

Segonne, Fernando Sanchez Vega, Alessandro Raganato, Jindřich Libovický, Jussi Karlgren, Shaoxiong Ji, Jindřich Helcl, Liane Guillou, Ona De Gibert, Jaione Bengoetxea, Joseph Attieh, and Marianna Apidianaki. 2025. [SemEval-2025 task 3: MuSHROOM, the multilingual shared-task on hallucinations and related observable overgeneration mistakes](#). In *Proceedings of the 19th International Workshop on Semantic Evaluation (SemEval-2025)*, pages 2472–2497, Vienna, Austria. Association for Computational Linguistics.

A Prompt Templates

We release the exact prompt code with the dataset. This appendix gives the main templates used for detector training and synthetic data construction. Variable slots are shown as {name}; lines beginning with ## are annotations in the figure, not part of the prompt.

```

You are an expert annotator who identifies hallucinated spans in a generated answer
with respect to a given context (the only trusted evidence). A hallucinated span is a
substring of the answer that is not supported by the context. Spans consistent with the
context are not hallucinations.

Quote each hallucinated span verbatim from the answer and classify it into exactly one
category and one subcategory.

Categories:
- contradiction: conflicts with the context (a wrong value, number, date, name, or relationship)
- fabricated_reference: an entity, name, identifier, or section that is absent from the context
- unsupported_addition: a claim, detail, or behavior the context never states

Subcategories:
entity, temporal, numerical, value, relational, identifier, section, attribute,
claim, behavior, elaboration, subjective, unspecified

Reply with ONLY a JSON object:
{"hallucinated_spans": [{"text": "...", "category": "...", "subcategory": "...}]}].
If nothing is unsupported, reply {"hallucinated_spans": []}.

User message:
{request_and_context}

Answer to verify:
{answer}

```

Figure 4: Detector prompt used for generative detector training and zero-shot LLM-judge evaluation. For code-agent rows in the generative detector training set, the same prompt also requests a short explanation field for each span; clean and hallucinated rows from that source both use the explanation variant, so the prompt choice does not leak the label.

```

## Question generation for README and Wikipedia sources
You generate a single, natural information-seeking question that can be answered
from a given document.

Document:
{document_chunk}

Generate one {question_type} question ({question_type_definition}) that the
document answers.

Rules:
1. Return ONLY the question, nothing else.
2. Use neutral, self-contained phrasing.
3. Keep it short and natural, like a query typed into a search engine.
4. It must be answerable from the document above.

## Clean-answer generation for tool-output, ACL, README, and Wikipedia sources
You are a helpful assistant answering a user's question using ONLY the provided
evidence. Write a correct, natural answer grounded strictly in that evidence.

Your answer MUST:
- Be accurate and fully supported by the evidence -- invent nothing.
- Reference concrete details from the evidence where relevant.
- Be concise.

Your answer must NOT:
- Add claims, identifiers, or values not present in the evidence.
- Include filler.

```

Figure 5: Question and clean-answer generation prompts. README and Wikipedia examples first generate a self-contained question from a markdown chunk; tool-output, ACL, README, and Wikipedia examples then generate a clean answer grounded only in the supplied evidence.

```
You are a hallucination injector for building a hallucination detection dataset.

You are given a CORRECT answer and the CONTEXT it is grounded in. Return ONLY a
small set of localized replacement edits that turn the answer into one containing
a specific kind of hallucination. The pipeline applies your edits; outside them
the answer must stay identical.

Target hallucination:
- Category: {category}
- Subtype: {subcategory}

CRITICAL grounding rule:
- The injected error MUST be detectable by comparing the answer against the
  provided context alone.

Rules:
- Make 1-2 distinct edits targeting the subtype above.
- Each replacement span must be as small as possible.
- Total changed text must be less than 30% of the answer.
- Changes must be plausible and subtle.
- Each "original" must be an exact substring of the answer, appearing exactly once.

Respond in JSON:
{"changes": [{"original": "...", "hallucinated": "...", "explanation": "...}]}
```

Figure 6: Generic injection prompt. The model proposes structured replacement edits; the pipeline applies them deterministically and recovers exact character offsets. Code uses two source-specific variants, one for wrong implementations and unrequested changes and one for fabricated methods, attributes, and keyword arguments. Both keep the same original/hallucinated output format.